Claude Code 中文全面教程（从入门到进阶）

Claude Code 是 Anthropic 推出的终端级 AI 编程代理工具，能够直接在命令行中实现代码生成、调试修复、项目导航、任务自动化等功能，堪称程序员的“效率倍增器”。本教程将从环境准备到进阶技巧，带你系统掌握 Claude Code 的核心用法，全程中文讲解，新手也能轻松上手。

一、前置准备

在安装 Claude Code 前，需完成以下基础环境配置，确保工具能正常运行。

1.1 核心依赖工具

- **Node.js 或 Bun**：Claude Code 基于 Node 开发，需安装 Node.js 18+（推荐 LTS 20+ 版本）或 Bun 运行环境。
- **Git**：用于代码管理，Claude Code 会借助 Git 读取项目历史、生成差异文件等。
- **科学网络环境**：由于 Claude Code 依赖 Anthropic 服务，需确保网络可正常访问外部资源。

1.2 依赖工具安装验证

安装完成后，可通过以下命令验证是否安装成功（打开终端/命令提示符输入）：

Code block

```
# 验证 Node.js 版本（Bun 用户输入 bun -v）
node -v
# 验证 Git 版本
git --version
```

若终端输出版本号，说明依赖工具安装成功。

二、安装与配置 Claude Code

完成前置准备后，即可进行 Claude Code 的安装和核心配置。

2.1 安装 Claude Code

通过 npm 全局安装（Windows、macOS、Linux 通用命令）：

Code block

```
npm install -g @anthropic-ai/claude-code
```

若不想全局安装，也可通过 npx 临时运行：

```
Code block
1   npx @anthropic-ai/claude-code
```

安装完成后，验证版本：

```
Code block
1   claude --version
```

输出版本信息即代表安装成功。

2.2 模型配置（关键步骤）

Claude Code 需配置大模型才能工作，推荐国内用户使用国产模型（如 GLM-4.7、M2.1），操作更便捷。以下以 GLM-4.7 为例，讲解配置流程：

1. 获取 API Key：前往 智谱开放平台 注册账号，创建应用后获取 API Key（需妥善保管，切勿泄露）。
2. 配置环境变量：
   - Windows 系统（命令提示符中输入）：
     `setx ANTHROPIC_BASE_URL "https://open.bigmodel.cn/api/anthropic"`
     `setx ANTHROPIC_AUTH_TOKEN "你的智谱API Key"`
     `setx ANTHROPIC_MODEL "GLM-4.7"`
   - macOS/Linux 系统（终端中输入）：
     `export ANTHROPIC_BASE_URL=https://open.bigmodel.cn/api/anthropic`
     `export ANTHROPIC_AUTH_TOKEN=你的智谱API Key`
     `export ANTHROPIC_MODEL=GLM-4.7`
3. 配置生效：重新打开终端，输入 `claude` 启动工具，若能正常进入交互界面，说明配置成功。

三、基础使用：快速上手

Claude Code 核心操作均在终端完成，掌握以下基础用法，即可快速开展工作。

3.1 启动与退出

- 启动：打开终端，输入 `claude` 后回车，进入交互模式（终端会显示 Claude Code 标识）。

- 退出：交互模式下，按 `Ctrl + C` 或输入 `/quit` 命令退出。

3.2 核心交互方式

Claude Code 支持自然语言交互，无需记忆复杂命令，核心交互逻辑为“描述需求 + 补充上下文”，示例：

Code block

```
# 基础需求：生成代码
你：用 JavaScript 写一个计算斐波那契数列的函数，要求支持任意长度输入

# 补充上下文的精准需求
你：我正在开发一个 React 项目，技术栈是 React + TypeScript + Ant Design，帮我写一个用户登录表单组件，包含账号密码验证和表单提交逻辑
```

3.3 常用斜杠命令

交互模式下，输入 `/` 可查看所有内置命令，常用命令如下：

- `/help`：查看帮助文档，获取命令说明和使用示例。
- `/memory`：打开记忆文件编辑界面，配置项目规范、个人偏好等（后续详细讲解）。
- `/resume`：恢复历史会话，可通过上下键选择需要恢复的会话记录。
- `/continue`：自动继续最近的一次会话。
- `/show`：查看当前项目中的文件列表或指定文件内容。

3.4 基础交互技巧

- 需求具体化：避免模糊表述，例如不说“修复这个漏洞”，而说“修复用户登录时不输入密码出现的空指针错误”。
- 复杂任务拆分：大需求拆分为小步骤，例如“创建电商订单系统”可拆分为“生成数据模型→编写 CRUD API→创建前端列表页面”。
- 引用文件/目录：使用 `@文件名` 或 `@目录名` 引用资源，例如“分析 @src/utils/date.ts 文件的重复逻辑并重构”。

四、核心功能详解

Claude Code 不仅能写代码，还能实现调试、导航、自动化等全流程开发任务，以下是五大核心功能的实战用法。

4.1 代码生成：从需求到代码一键生成

核心能力：通过自然语言描述功能需求，Claude Code 会自动制定实现方案、编写符合规范的代码，并考虑边界情况。

实战示例：

Code block

```
你：用 TypeScript 写一个用户管理模块的 API 接口，包含用户列表查询（支持分页）、用户新增、用户删除功能，要求使用 Express 框架，错误处理采用 try-catch 模式，返回统一的 JSON 格式。

Claude Code 输出：
1. 先给出实现思路（接口设计、参数定义、错误处理方案）
2. 生成完整的代码文件（包含路由定义、控制器逻辑、类型定义）
3. 给出调用示例和测试建议
```

4.2 调试修复：精准定位并解决问题

核心能力：粘贴错误信息或描述问题现象，Claude Code 会分析代码库、定位问题根源，并提供修复方案和修改后的代码。

实战示例：

Code block

```
你：以下是运行代码时的错误信息：
TypeError: Cannot read properties of undefined (reading 'name')
文件路径：src/components/UserCard.tsx
请帮我分析问题并修复。

Claude Code 输出：
1. 分析错误原因（可能是未判断用户数据是否存在）
2. 定位错误代码位置（指出 UserCard.tsx 中访问 user.name 的行数）
3. 提供修复方案（添加空值判断）
4. 生成修改后的完整代码
```

4.3 代码导航：快速掌握项目结构

核心能力：询问项目相关问题（如架构设计、文件功能、依赖关系），Claude Code 会基于整个项目结构给出清晰答案，无需手动翻阅文件。

实战示例：

Code block

```
你：分析当前项目的目录结构，说明每个目录的作用，以及核心业务逻辑在哪个文件中实现。

```

```
Claude Code 输出：
1. 列出项目目录树（如 src/app、src/components、src/api 等）
2. 逐一说明每个目录的功能定位
3. 指出核心业务逻辑所在的文件（如 src/services 目录下的用户服务、订单服务）
4. 给出快速熟悉项目的建议（优先查看哪些文件）
```

4.4 自动化任务：解放重复劳动

核心能力：自动化处理繁琐任务，如修复 lint 错误、解决合并冲突、生成发布说明、批量修改代码等。

实战示例：

Code block

```
你：帮我解决当前分支的 Git 合并冲突，冲突文件是 src/utils/format.ts，保留双方的核心逻辑，确保合并后代码可正常运行。

你：批量修改项目中所有文件的编码规范：将所有 var 关键字替换为 let/const，函数声明改为箭头函数（符合 ES6+ 标准）。
```

4.5 多工具集成：连接外部资源

核心能力：通过 MCP（Multipurpose Connector Protocol）连接 Google Drive、Figma、Slack 等外部工具，实现数据提取、文件读写、跨工具协作等功能。

实战示例：

Code block

```
你：从我的 Google Drive 文档（链接：xxx）中提取用户需求清单，生成对应的项目开发任务列表，并创建一个 Markdown 文档保存到当前项目的 docs 目录下。
```

五、进阶技巧：效率翻倍

掌握以下进阶技巧，能让 Claude Code 的使用效率再上一个台阶，适合中高级用户。

5.1 记忆管理：定制专属开发规范

Claude Code 会自动读取项目中的记忆文件，用于存储项目规范、个人偏好等上下文信息，无需每次重复说明。

- 记忆文件类型及用途：
  - 项目共享记忆（./CLAUDE.md）：团队共享的编码规范、项目架构、工作流程等。

    - 用户全局记忆（~/.claude/CLAUDE.md）：个人通用偏好（如默认中文回复、代码风格）。
- 编辑记忆文件：交互模式下输入 `/memory`，选择需要编辑的记忆文件，添加自定义规则，例如：

```
# 个人偏好
1. 所有交互均使用中文回复
2. 生成代码时默认添加详细注释
3. 优先使用 TypeScript 类型推导

# 项目编码规范
1. 采用函数组件和 React Hooks，不使用类组件
2. API 请求统一使用项目封装的 apiClient 函数
3. 错误处理需包含用户友好的提示信息
```

5.2 批量重构：快速优化代码结构

针对大型项目的批量修改需求，可通过清晰的指令让 Claude Code 自动完成重构，避免手动操作的遗漏和错误。

实战示例：

Code block

```
你：重构项目中的 API 命名规范：
1. 所有 GET 请求函数改为 fetch* 前缀（如 getUser → fetchUser）
2. 所有 POST 请求改为 create* 前缀（如 addOrder → createOrder）
3. 所有 PUT 请求改为 update* 前缀
4. 所有 DELETE 请求改为 remove* 前缀
要求更新所有相关文件（包括类型定义、测试用例、文档注释），确保代码可正常运行。
```

5.3 上下文链式对话：逐步细化需求

Claude Code 会记住整个对话历史，可通过多轮对话逐步细化需求，无需一次性完整描述。

实战示例：

Code block

```
你：我正在开发一个电商后台管理系统，技术栈是 Next.js + TypeScript + Prisma + Ant Design。
Claude：已记录技术栈信息，等待后续需求。

你：创建商品列表页面。
Claude：生成基础商品列表页面代码（符合技术栈规范）。

你：在这个页面添加实时搜索功能，带防抖效果（延迟 500ms）。
Claude：在现有代码基础上添加搜索逻辑和防抖处理。
```

```

你：再添加高级筛选功能，支持按商品分类、价格区间、库存状态筛选。
Claude：继续优化页面，添加筛选组件和筛选逻辑。
```

5.4 深度思考模式：解决复杂问题

针对复杂需求，可使用 `think` 系列指令激活深度思考模式，提升 Claude Code 的分析和解决能力（注意：高级模式会消耗更多资源）。

- 模式级别（从弱到强）：think < think hard < think harder < ultrathink
- 使用示例：
  `你：设计一个高并发的订单支付系统，需要考虑分布式锁、事务一致性、峰值流量处理，ultrathink`

六、常见问题与解决方案

汇总使用过程中常见的问题及解决方法，帮助你快速排障。

6.1 “claude 不是内部或外部命令”

原因：npm 全局 bin 目录未添加到系统环境变量 PATH 中。

解决方案：

1. 执行 `npm config get prefix` 获取 npm 全局目录。
2. 将该目录下的 `node_modules\.bin` 路径添加到系统环境变量 PATH 中。
3. 重新打开终端验证。

6.2 网络超时/无法连接服务

原因：网络环境无法访问 Anthropic 或配置的模型服务。

解决方案：

- 检查科学网络环境是否正常。
- 确认环境变量中配置的 BASE_URL 和 API Key 正确。
- 尝试切换其他可访问的模型（如 M2.1、K2）。

6.3 权限不足，无法读写文件

解决方案：

- Windows：以管理员身份运行命令提示符。
- macOS/Linux：在命令前添加 `sudo`（如 `sudo claude`），或修改项目目录权限。
- 避免将项目放在中文路径或含空格的路径下。

6.4 生成代码不符合预期

解决方案：

- 细化需求描述，补充更多上下文（如技术栈、功能边界、编码规范）。
- 通过 `/memory` 配置项目规范，让 Claude 自动遵循。
- 使用链式对话逐步修正，例如“生成的代码缺少 X 功能，帮我补充”。

七、总结与拓展资源

本教程覆盖了 Claude Code 的核心用法，从环境配置到进阶技巧，足够支撑日常开发需求。想要进一步提升，可参考以下资源：

- 官方文档：Claude Code 官方中文文档
- 进阶功能：MCP 配置、Plugins 开发、SubAgents 并行处理等高级特性。
- 社区资源：稀土掘金、51CTO 等平台的 Claude Code 实战案例分享。

Claude Code 的核心价值是“解放重复劳动，聚焦核心逻辑”，建议在实际项目中多尝试、多调整，逐步找到适合自己的使用方式，最大化提升开发效率。

（注：文档部分内容可能由 AI 生成）